

CHAPTER 7

Backup and Restore in PostgreSQL

Introduction

One of the important aspects of administering the database is to take regular backups and test the same by restoring the same. At any point in time, DBA might need to restore the previous dates' backups, which are important to perform the point-in-time recovery and used to roll back the accidental changes done to the database. Backups are extremely important from the high availability standpoint as well. They can build or rebuild standby servers as an initial copy.

Structure

In this chapter, we will learn how to take a backup of the PostgreSQL database and how to restore the same. This chapter is going to cover the following concepts:

- Backup
- Restore
- Useful Backup and restore tools

Objectives

In this chapter you will learn about the different types of backups you can take from your PostgreSQL database, their differences, and how to restore them. Knowing this is essential to build your path as PostgreSQL DBA. Also, we will review a few of the most used backup tools provided by the community, which provide extra functionalities and enhancements.

Backup

Database should be backed up at regular intervals, which can help in maintaining the copy of the data. In case any unexpected changes happen to the database can be reverted by restoring the backup. Backup can also help in performing point-in-time recovery if needed. Backups are important to build / rebuild the slave databases too.

As shown in *Figure 7.1*, please find the different backup types and the tools given by the community for the same:

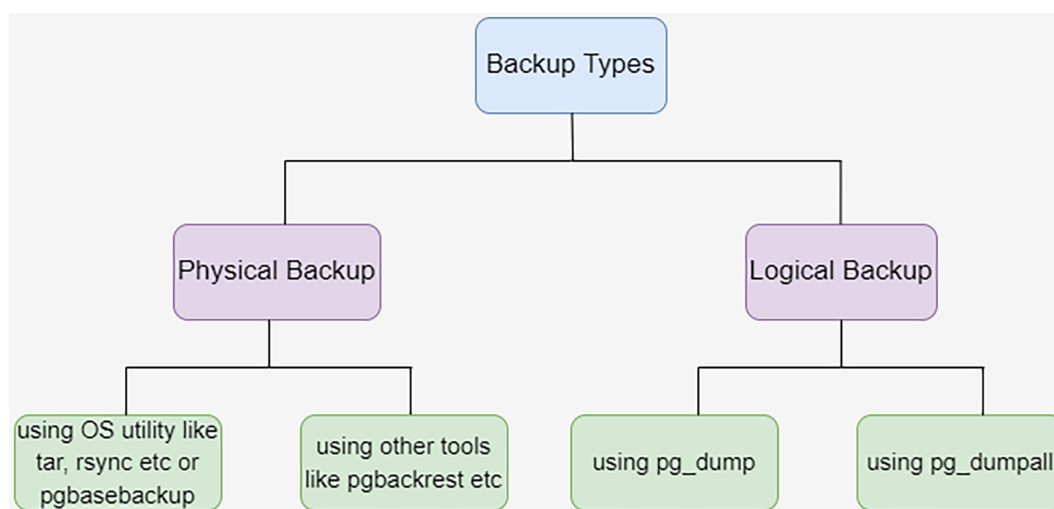


Figure 7.1: Backup Types

Physical backup

Physical backups are filesystem-level backups that can be taken, by copying the data directory along with the custom tablespace directory from one server to another server. The only challenge here is to take backup consistently. However, one can take backup at the cluster level, and single DB object level backup like single DB level or table level backup is not possible. As far as consistency of the backup is concerned, one can adhere to the high-level steps to take physical backup as following.

1. Create a checkpoint using the **CHECKPOINT** command and **pg_start_backup** ('any_label_name') command to tell the database that it has to issue a logical end to ongoing transactions. Basically, it will switch the **Write Ahead Log (WAL)** files and keep a mark internally saying that backup has been taken till the marked point.
2. Copy the physical data directory along with tablespace directory (if created outside data directory) using any OS utility like **tar** or **rsync**
3. Issue **pg_stop_backup()** command once the copy is completed as mentioned in step 2.

pg_basebackup

A utility given by PostgreSQL community to take backup of the whole PostgreSQL database cluster. (Reference: **pg_basebackup**)

Following is the simplest example of how the **pg_basebackup** utility can be used to take a backup of the data directory using the “-D” option

```
postgres@ip-172-31-27-8:~$ pg_basebackup -U postgres -h localhost -x  
-D /usr/local/pgsql/data
```

Figure 7.2 shows the various options which can be used with the **pg_basebackup**. Note, how the OS help command has been used to fetch the usage of the command.

```

postgres@ip-172-31-27-8:~$ pg_basebackup --help
pg_basebackup takes a base backup of a running PostgreSQL server.

Usage:
  pg_basebackup [OPTION]...

Options controlling the output:
  -D, --pgdata=DIRECTORY  receive base backup into directory
  -F, --format=plt         output format (plain (default), tar)
  -r, --max-rate=RATE      maximum transfer rate to transfer data directory
                          (in kB/s, or use suffix "k" or "M")
  -R, --write-recovery-conf
                          write configuration for replication
  -T, --tablespace-mapping=OLDDIR=NEWDIR
                          relocate tablespace in OLDDIR to NEWDIR
  --waldir=WALDIR          location for the write-ahead log directory
  -X, --wal-method=none|fetch|stream
                          include required WAL files with specified method
  -z, --gzip               compress tar output
  -Z, --compress=0-9       compress tar output with given compression level

General options:
  -c, --checkpoint=fast|spread
                          set fast or spread checkpointing
  -C, --create-slot        create replication slot
  -l, --label=LABEL        set backup label
  -n, --no-clean           do not clean up after errors
  -N, --no-sync            do not wait for changes to be written safely to disk
  -P, --progress           show progress information
  -S, --slot=SLOTNAME      replication slot to use
  -v, --verbose            output verbose messages
  -V, --version            output version information, then exit
  --manifest-checksums=SHA{224,256,384,512}|CRC32C|NONE
                          use algorithm for manifest checksums
  --manifest-force-encode
                          hex encode all file names in manifest
  --no-estimate-size       do not estimate backup size in server side
  --no-manifest            suppress generation of backup manifest
  --no-slot               prevent creation of temporary replication slot
  --no-verify-checksums
                          do not verify checksums
  -?, --help              show this help, then exit

Connection options:
  -d, --dbname=CONNSTR     connection string
  -h, --host=HOSTNAME      database server host or socket directory
  -p, --port=PORT          database server port number
  -s, --status-interval=INTERVAL
                          time between status packets sent to server (in seconds)
  -U, --username=NAME      connect as specified database user
  -w, --no-password        never prompt for password
  -W, --password           force password prompt (should happen automatically)

Report bugs to <pgsql-bugs@lists.postgresql.org>.
PostgreSQL home page: <https://www.postgresql.org/>
postgres@ip-172-31-27-8:~$

```

Figure 7.2: Options in `pg_basebackup`

Point in time Recovery/Archival

Point in Time Recovery, or PITR in short, can also be achieved in postgres by enabling archiving. In simple words, one can restore the database to a specific time and recover the database. To do the same, WAL/archives must be restored till the specific time frame.

The following parameters must be enabled in **postgresql.conf** file to achieve PITR: (Reference: Point In Time Recovery)

```
archive_mode=on
```

```
archive_command = 'test ! -f /mnt/server/archivedir/%f && cp %p /mnt/
server/archivedir/%f' # Unix
```

```
archive_command = 'copy "%p" "C:\\server\\archivedir\\%f"' # Windows
```

We will see in the restore section later in this chapter on how to restore the database to a specific time frame.

Pros and cons of physical backup

- The major advantage is backup contains a filesystem-level backup of the data directory, which in turn also contains the backup of configuration files like **postgresql.conf**, **pg_hba.conf**, and the like. It is as simple as copying the data directory along with custom tablespaces to the destination server and starting the postgres service.
- The major disadvantage is that it can only take cluster-level backup. Backup for individual DB or single table is not possible.
- It is difficult to find the consistency of the backup.
- The speed of the data to be copied depends on the server hardware and the network speed (in case data is copied over the network to a remote server).

Logical backup

Logical backup is used to take a backup of the database in a plain human-readable format. It can also be used to take backup of the specific DB object, like only a particular DB from the whole DB cluster or the single/multiple tables. The PostgreSQL community has given multiple utilities which can help in taking logical backups of the PostgreSQL DB. Let us try to dig deeper into the most frequently used commands one by one.

pg_dump

pg_dump is one of the most common logical backup utilities that help take backup of a single database. It also can take backup of single or multiple tables and sequences. One can take backup in plain text format and compressed format.

pg_dump can also be used as a tool to export specific DB Objects in “**.sql**” file. File extracted in **.sql** format can be imported using the **psql** command. Any file which has been exported in custom or compressed format can be restored using the **pg_restore** command. Also, it is important to note that this lightweight utility neither blocks readers nor writers while taking backup of any object.

pg_dumpall

pg_dumpall utility is used to take logical backup of the whole database cluster, unlike **pg_dump**, which takes backup of only specific DB or specific DB Object. This utility is majorly used to export cluster-level objects like one can generate a script of global objects - tablespaces, users, and the like. It works exactly the same as **pg_dump**, except it works for cluster-level scripts/backups. custom/compressed format backups can be restored using **pg_restore**, and **.sql** (plain text file) format files can be executed using the **psql** command.

Figure 7.3 shows the options which can be used with **pg_dump**.

```

postgres@ip-172-31-27-8:~$ pg_dump --help
pg_dump dumps a database as a text file or to other formats.

Usage:
  pg_dump [OPTION]... [DBNAME]

General options:
  -f, --file=FILENAME          output file or directory name
  -F, --format=cldt|lp         output file format (custom, directory, tar,
                                plain text (default))
  -j, --jobs=NUM               use this many parallel jobs to dump
  -v, --verbose                 verbose mode
  -V, --version                 output version information, then exit
  -Z, --compress=0-9           compression level for compressed formats
  --lock-wait-timeout=TIMEOUT  fail after waiting TIMEOUT for a table lock
  --no-sync                     do not wait for changes to be written safely to disk
  -?, --help                   show this help, then exit

Options controlling the output content:
  -a, --data-only              dump only the data, not the schema
  -b, --blobs                  include large objects in dump
  -B, --no-blobs               exclude large objects in dump
  -c, --clean                  clean (drop) database objects before recreating
  -C, --create                 include commands to create database in dump
  -e, --extension=PATTERN     dump the specified extension(s) only
  -E, --encoding=ENCODING     dump the data in encoding ENCODING
  -n, --schema=PATTERN        dump the specified schema(s) only
  -N, --exclude-schema=PATTERN do NOT dump the specified schema(s)
  -O, --no-owner               skip restoration of object ownership in
                                plain-text format
  -s, --schema-only            dump only the schema, no data
  -S, --superuser=NAME        superuser user name to use in plain-text format
  -t, --table=PATTERN          dump the specified table(s) only
  -T, --exclude-table=PATTERN do NOT dump the specified table(s)
  -x, --no-privileges          do not dump privileges (grant/revoke)
  --binary-upgrade             for use by upgrade utilities only
  --column-inserts             dump data as INSERT commands with column names
  --disable-dollar-quoting     disable dollar quoting, use SQL standard quoting
  --disable-triggers           disable triggers during data-only restore
  --enable-row-security        enable row security (dump only content user has
                                access to)
  --exclude-table-data=PATTERN do NOT dump data for the specified table(s)
  --extra-float-digits=NUM     override default setting for extra_float_digits
  --if-exists                  use IF EXISTS when dropping objects
  --include-foreign-data=PATTERN
                                include data of foreign tables on foreign
                                servers matching PATTERN
  --inserts                    dump data as INSERT commands, rather than COPY
  --load-via-partition-root    load partitions via the root table
  --no-comments                do not dump comments
  --no-publications            do not dump publications
  --no-security-labels         do not dump security label assignments
  --no-subscriptions           do not dump subscriptions
  --no-synchronized-snapshots do not use synchronized snapshots in parallel jobs
  --no-tablespaces             do not dump tablespace assignments
  --no-toast-compression       do not dump TOAST compression methods
  --no-unlogged-table-data     do not dump unlogged table data
  --on-conflict-do-nothing     add ON CONFLICT DO NOTHING to INSERT commands
  --quote-all-identifiers     quote all identifiers, even if not key words
  --rows-per-insert=NROWS     number of rows per INSERT; implies --inserts
  --section=SECTION            dump named section (pre-data, data, or post-data)
  --serializable-deferrable    wait until the dump can run without anomalies
  --snapshot=SNAPSHOT          use given snapshot for the dump
  --strict-names               require table and/or schema include patterns to
                                match at least one entity each
  --use-set-session-authorization
                                use SET SESSION AUTHORIZATION commands instead of
                                ALTER OWNER commands to set ownership

Connection options:
  -d, --dbname=DBNAME         database to dump
  -h, --host=HOSTNAME         database server host or socket directory
  -p, --port=PORT              database server port number
  -U, --username=NAME          connect as specified database user
  -w, --no-password            never prompt for password
  -W, --password               force password prompt (should happen automatically)
  --role=ROLENAME              do SET ROLE before dump

If no database name is supplied, then the PGDATABASE environment
variable value is used.

Report bugs to <pgsql-bugs@lists.postgresql.org>.
PostgreSQL home page: <https://www.postgresql.org/>
postgres@ip-172-31-27-8:~$

```

Figure 7.3: Options in `pg_dump`

Figure 7.4 shows the options used in the `pg_dumpall` command:

```
postgres@ip-172-31-27-8:~$ pg_dumpall --help
pg_dumpall extracts a PostgreSQL database cluster into an SQL script file.

Usage:
  pg_dumpall [OPTION]...

General options:
  -f, --file=FILENAME      output file name
  -v, --verbose             verbose mode
  -V, --version            output version information, then exit
  --lock-wait-timeout=TIMEOUT fail after waiting TIMEOUT for a table lock
  -?, --help              show this help, then exit

Options controlling the output content:
  -a, --data-only          dump only the data, not the schema
  -c, --clean              clean (drop) databases before recreating
  -E, --encoding=ENCODING dump the data in encoding ENCODING
  -g, --globals-only       dump only global objects, no databases
  -O, --no-owner           skip restoration of object ownership
  -r, --roles-only         dump only roles, no databases or tablespaces
  -s, --schema-only        dump only the schema, no data
  -S, --superuser=NAME     superuser user name to use in the dump
  -t, --tablespaces-only   dump only tablespaces, no databases or roles
  -x, --no-privileges      do not dump privileges (grant/revoke)
  --binary-upgrade         for use by upgrade utilities only
  --column-inserts         dump data as INSERT commands with column names
  --disable-dollar-quoting disable dollar quoting, use SQL standard quoting
  --disable-triggers       disable triggers during data-only restore
  --exclude-database=PATTERN exclude databases whose name matches PATTERN
  --extra-float-digits=NUM override default setting for extra_float_digits
  --if-exists              use IF EXISTS when dropping objects
  --inserts                dump data as INSERT commands, rather than COPY
  --load-via-partition-root load partitions via the root table
  --no-comments            do not dump comments
  --no-publications        do not dump publications
  --no-role-passwords      do not dump passwords for roles
  --no-security-labels     do not dump security label assignments
  --no-subscriptions       do not dump subscriptions
  --no-sync                do not wait for changes to be written safely to disk
  --no-tablespaces         do not dump tablespace assignments
  --no-toast-compression   do not dump TOAST compression methods
  --no-unlogged-table-data do not dump unlogged table data
  --on-conflict-do-nothing add ON CONFLICT DO NOTHING to INSERT commands
  --quote-all-identifiers quote all identifiers, even if not key words
  --rows-per-insert=NROWS  number of rows per INSERT; implies --inserts
  --use-set-session-authorization use SET SESSION AUTHORIZATION commands instead of
                              ALTER OWNER commands to set ownership

Connection options:
  -d, --dbname=CONNSTR    connect using connection string
  -h, --host=HOSTNAME      database server host or socket directory
  -l, --database=DBNAME    alternative default database
  -p, --port=PORT         database server port number
  -U, --username=NAME      connect as specified database user
  -w, --no-password        never prompt for password
  -W, --password           force password prompt (should happen automatically)
  --role=ROLENAME          do SET ROLE before dump

If -f/--file is not used, then the SQL script will be written to the standard
output.

Report bugs to <pgsql-bugs@lists.postgresql.org>.
PostgreSQL home page: <https://www.postgresql.org/>
postgres@ip-172-31-27-8:~$
```

Figure 7.4: Options in `pg_dumpall`

Pros and cons of logical backup

- Specific parts of the database can be backed up with the help of logical backup.
- It is a simple and human-readable backup.
- Since it allows taking the backup in the plain text/SQL file, backup taken in one OS can easily be restored in another OS with the least minimal issues, unlike Physical backup.
- One needs to take a backup of configuration files separately as logical backup does not include the same as physical backup.
- Since a single DB object can be taken, it is faster than a physical backup.
- PITR is not supported in Logical backup.

Restore

Taking a backup is less than half of the job done. An important aspect of the backup is that it should get successfully restored. Backups can also be used to perform PITR or create copies of the database in another server and so on.

Same as backups, there are two different types of restore, as mentioned in *Figure 7.5*:

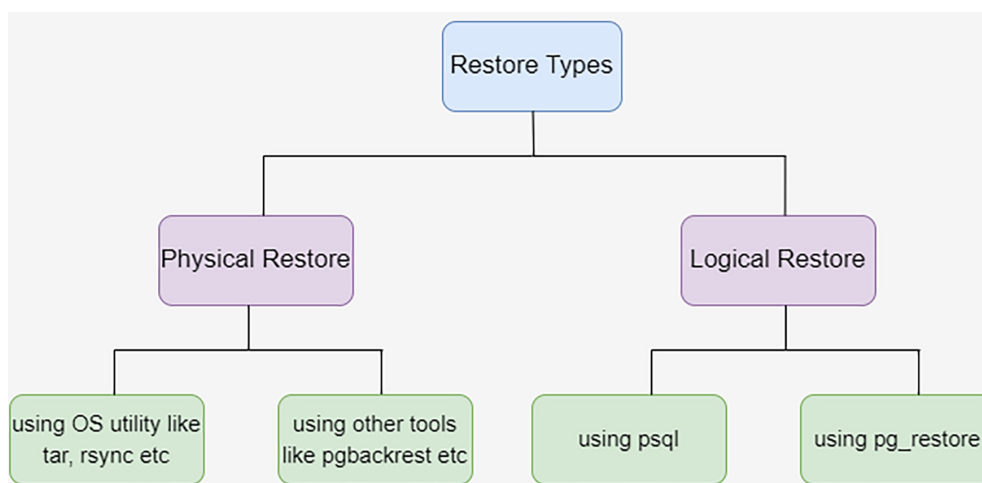


Figure 7.5: Restore Types

psql

psql command is used to restore the backup, which is taken in “.sql” format or plain text format. This utility can be compared to the *import* utility of any other

RDBMS, which helps execute the “.sql” files. The other use of **psql** is to log in to the **postgres** database.

Figure 7.6 shows the various options which **psql** command offers:

```
ubuntu@ip-172-31-28-98:~$ psql --help
psql is the PostgreSQL interactive terminal.

Usage:
  psql [OPTION]... [DBNAME [USERNAME]]

General options:
  -c, --command=COMMAND      run only single command (SQL or internal) and exit
  -d, --dbname=DBNAME        database name to connect to (default: "ubuntu")
  -f, --file=FILENAME        execute commands from file, then exit
  -l, --list                  list available databases, then exit
  -v, --set=, --variable=NAME=VALUE
                              set psql variable NAME to VALUE
                              (e.g., -v ON_ERROR_STOP=1)
  -V, --version               output version information, then exit
  -X, --no-psqlrc             do not read startup file (~/.psqlrc)
  -1 ("one"), --single-transaction
                              execute as a single transaction (if non-interactive)
  -?, --help[=options]       show this help, then exit
  --help=commands             list backslash commands, then exit
  --help=variables            list special variables, then exit

Input and output options:
  -a, --echo-all             echo all input from script
  -b, --echo-errors           echo failed commands
  -e, --echo-queries          echo commands sent to server
  -E, --echo-hidden          display queries that internal commands generate
  -L, --log-file=FILENAME     send session log to file
  -n, --no-readline           disable enhanced command line editing (readline)
  -o, --output=FILENAME       send query results to file (or lpipe)
  -q, --quiet                 run quietly (no messages, only query output)
  -s, --single-step           single-step mode (confirm each query)
  -S, --single-line           single-line mode (end of line terminates SQL command)

Output format options:
  -A, --no-align              unaligned table output mode
  --csv                       CSV (Comma-Separated Values) table output mode
  -F, --field-separator=STRING
                              field separator for unaligned output (default: "|")
  -H, --html                  HTML table output mode
  -P, --pset=VAR[=ARG]        set printing option VAR to ARG (see \pset command)
  -R, --record-separator=STRING
                              record separator for unaligned output (default: newline)
  -t, --tuples-only           print rows only
  -T, --table-attr=TEXT       set HTML table tag attributes (e.g., width, border)
  -x, --expanded              turn on expanded table output
  -z, --field-separator-zero
                              set field separator for unaligned output to zero byte
  -0, --record-separator-zero
                              set record separator for unaligned output to zero byte

Connection options:
  -h, --host=HOSTNAME         database server host or socket directory (default: "/var/run/postgresql")
  -p, --port=PORT             database server port (default: "5432")
  -U, --username=USERNAME     database user name (default: "ubuntu")
  -w, --no-password           never prompt for password
  -W, --password              force password prompt (should happen automatically)

For more information, type "\?" (for internal commands) or "\help" (for SQL
commands) from within psql, or consult the psql section in the PostgreSQL
documentation.

Report bugs to <pgsql-bugs@lists.postgresql.org>.
PostgreSQL home page: <https://www.postgresql.org/>
```

Figure 7.6: options in psql

pg_restore

pg_restore helps in restoring the logical backups taken in a custom format. Please find the attributes used by the **pg_restore** command in *Figure 7.7*:

```
postgres@ip-172-31-27-8:~$ pg_restore --help
pg_restore restores a PostgreSQL database from an archive created by pg_dump.

Usage:
  pg_restore [OPTION]... [FILE]

General options:
  -d, --dbname=NAME          connect to database name
  -f, --file=FILENAME        output file name (- for stdout)
  -F, --format=cldlt         backup file format (should be automatic)
  -l, --list                 print summarized TOC of the archive
  -v, --verbose              verbose mode
  -V, --version              output version information, then exit
  -?, --help                show this help, then exit

Options controlling the restore:
  -a, --data-only            restore only the data, no schema
  -c, --clean                clean (drop) database objects before recreating
  -C, --create               create the target database
  -e, --exit-on-error        exit on error, default is to continue
  -I, --index=NAME           restore named index
  -j, --jobs=NUM             use this many parallel jobs to restore
  -L, --use-list=FILENAME    use table of contents from this file for
                             selecting/ordering output
  -n, --schema=NAME          restore only objects in this schema
  -N, --exclude-schema=NAME  do not restore objects in this schema
  -O, --no-owner             skip restoration of object ownership
  -P, --function=NAME(args)  restore named function
  -s, --schema-only          restore only the schema, no data
  -S, --superuser=NAME       superuser user name to use for disabling triggers
  -t, --table=NAME           restore named relation (table, view, etc.)
  -T, --trigger=NAME         restore named trigger
  -x, --no-privileges        skip restoration of access privileges (grant/revoke)
  -1, --single-transaction   restore as a single transaction
  --disable-triggers         disable triggers during data-only restore
  --enable-row-security      enable row security
  --if-exists                use IF EXISTS when dropping objects
  --no-comments              do not restore comments
  --no-data-for-failed-tables do not restore data of tables that could not be
                             created
  --no-publications          do not restore publications
  --no-security-labels       do not restore security labels
  --no-subscriptions         do not restore subscriptions
  --no-tablespaces           do not restore tablespace assignments
  --section=SECTION          restore named section (pre-data, data, or post-data)
  --strict-names             require table and/or schema include patterns to
                             match at least one entity each
  --use-set-session-authorization
                             use SET SESSION AUTHORIZATION commands instead of
                             ALTER OWNER commands to set ownership

Connection options:
  -h, --host=HOSTNAME        database server host or socket directory
  -p, --port=PORT            database server port number
  -U, --username=NAME         connect as specified database user
  -w, --no-password          never prompt for password
  -W, --password             force password prompt (should happen automatically)
  --role=ROLENAME            do SET ROLE before restore

The options -I, -n, -N, -P, -t, -T, and --section can be combined and specified
multiple times to select multiple objects.

If no input file name is supplied, then standard input is used.

Report bugs to <pgsql-bugs@lists.postgresql.org>.
PostgreSQL home page: <https://www.postgresql.org/>
postgres@ip-172-31-27-8:~$
```

Figure 7.7: Options in **pg_restore**

Let us see few examples of logical backup and restore. Refer *Table 7.1*:

Description	Command
To dump a database called mydb into a SQL-script file:	<code>pg_dump mydb > "/opt/mydb_dump.sql"</code>
To reload such a script into a (freshly created) database named newdb:	<code>psql -d newdb -f "/opt/mydb_dump.sql"</code>
To dump a database into a custom-format archive file:	<code>pg_dump -Fc mydb > "/opt/mydb_dump.dump"</code>
To reload an archive file into a (freshly created) database named newdb:	<code>pg_restore -d newdb "/opt/mydb_dump.dump"</code>
To dump a single table named mytab	<code>pg_dump -t mytab mydb > "/opt/mydb_dump.sql"</code>
To dump the database with the log file using user test	<code>pg_dump -U test -v -f "/opt/mydb_dump.sql" mydb 2>> "/opt/mydb_log.log"</code> password for user test :
To dump only structure without data	<code>pg_dump -sU test -v -f "/opt/mydb_dump.sql" mydb 2>> "/opt/mydb_log.log"</code> password for user test :
To take insert scripts of particular table	<code>pg_dump --column-inserts -a -t mytab -U test mydb > "/opt/mytab_inserts.sql"</code>
To dump only specific tables with data	<code>pg_dump -U test -n schema1 -t BCL_* -f "/opt/BCL_TABLES.sql" mydb</code>

Table 7.1: Logical Backup and Restore example

Useful backup and restore tools

Now that we know the importance of backups when working with PostgreSQL and how they are taken and restored, we will review some of the most popular backup tools we can find and use.

PostgreSQL, directly out of the box, provides the necessary interfaces and functions to take any of the two types of backups, logical (**pg_dump**, **pg_dumpall**) and physical (**pg_basebackup**). Also, it is possible to perform continuous archiving (**archive_command**). However, due to the extraordinary community behind postgres and the continuous commitment to improving the existing options, we have many different tools that can help to create and manage better and more flexible backup strategies.

Some of these tools provide the next benefits:

- Full, incremental, and differential backups.
- Backup compression and checksum.
- Different backup repositories, such as on-prem storage and cloud repositories.
- Parallelism.
- Backups catalog.
- Alerting and notifications.

Then let us review some of the most useful open-source tools, their options, and how to perform a backup and restore it.

pgBackRest

This is one of the most popular and robust backup tools nowadays. You can configure it as a central repository to handle multiple databases backup. It can store the backups in a given path in the backup server or use a cloud bucket for the same; also, you can configure it to back up to both locations simultaneously.

With pgBackRest, you can perform **full, incremental, and differential** backups. You can set the backup operation to run with parallelism, so if your system has multiple CPUs, you can take advantage of them and speed up the backup and restore. Also, you can configure it to take the backup from a standby rather than the primary server, avoiding adding load to the primary server.

Additionally, the continuous archiving can be set as *asynchronous*, meaning pgBackRest will confirm the **postgres** database the WAL was successfully archived, but the actual archive (copying the WAL file to its final destination) happens sometime after. This improves the archive rate and releases some overhead from the database when the WAL creation rate is high due to the heavy workload.

Figure 7.8 shows an example considering two database servers and one backup server running **pgBackRest** and using a cloud bucket (AWS S3) as a repository to store the backups.

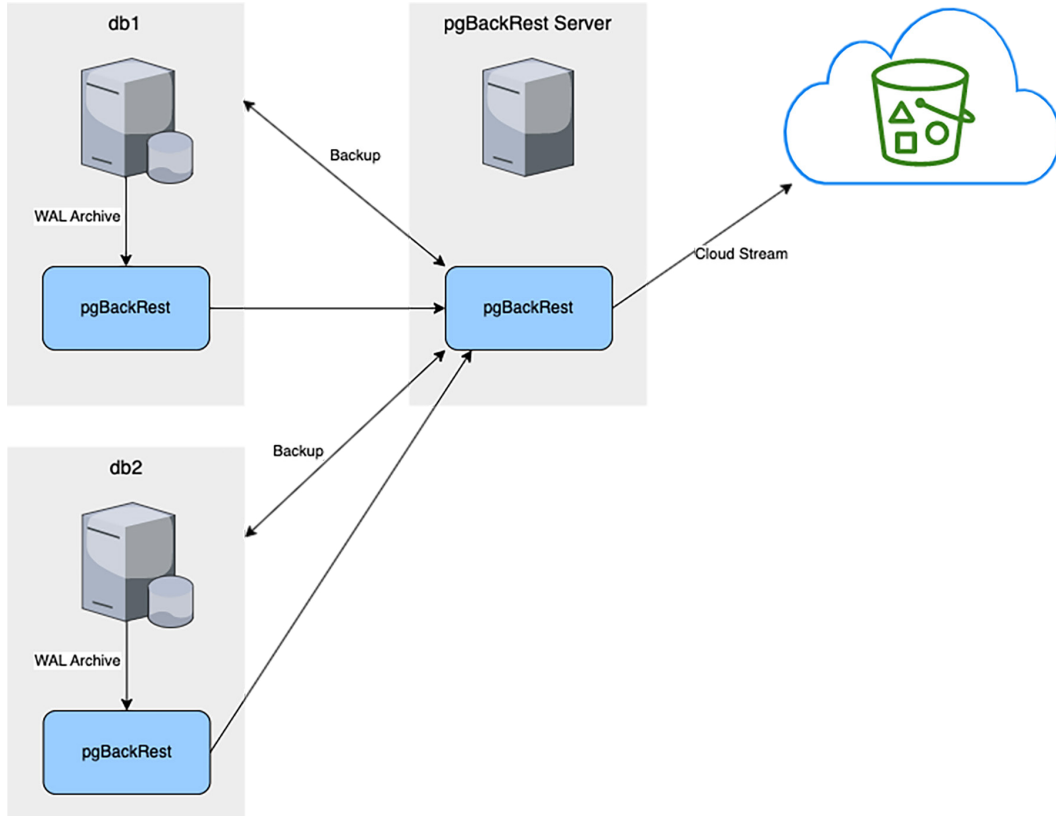


Figure 7.8: pgBackRest for multiple DB servers and using cloud storage

To get this set up working, it is necessary to configure **pgBackRest** in the backup server and the database server because the WAL archiving happens on the database side, so it is necessary to configure PostgreSQL to use **pgBackRest** when a WAL file needs to be archived.

The following is an example of what the **pgBackRest** configuration looks like for the setup in Figure 7.8. Remember, the password-less SSH should be configured between the **pgBackRest** server and the database servers:

```
postgres@repository:~$ ssh db1 'hostname'
db1
postgres@repository:~$ ssh db2 'hostname'
db2
```

pgBackRest server:

```
[root@repository ~]# cat /etc/pgbackrest.conf

[global]
repo1-path=/pgbackrest
repo1-retention-full=2
start-fast=y
repo1-type=s3
repo1-s3-bucket=pgbackrest-backups
repo1-s3-key=<aws access key>
repo1-s3-key-secret=<aws secret key>
repo1-s3-token=<aws session token>

[db1]
pg1-path=/var/lib/postgresql/13/main
pg1-host=db1
pg1-host-user=postgres

[db2]
pg1-path=/var/lib/postgresql/14/main
pg1-host=db2
pg1-host-user=postgres
```

db1 server:

```
[root@db1 ~]# cat /etc/pgbackrest.conf

[global]
log-level-file=detail
repo1-host=repository
repo1-host-user=postgres
```

```
[db1]
pg1-path=/var/lib/postgresql/13/main
```

db2 server:

```
[root@db2 ~]# cat /etc/pgbackrest.conf

[global]

log-level-file=detail

repo1-host=repository

repo1-host-user=postgres

[db2]

pg1-path=/var/lib/postgresql/14/main
```

Now you can create the *stanza*, which means the repository for the defined database. In the above example, since we will take backups from two different database servers, we would need to create two stanzas (*db1* and *db2*). Figure 7.9 illustrates the creation of the stanza for the *db1*:

```
postgres@repository:~$ pgbackrest --stanza=db1 --log-level-console=info stanza-create
2022-11-15 22:07:40.185 P00 INFO: stanza-create command begin 2.41: --exec-id=8303-e6b7ee75 --log-level-console=info --pg1-host=db1
--pg1-host-user=postgres --pg1-path=/var/lib/postgresql/13/main --repo1-path=/pgbackrest --repo1-s3-bucket=pgbackrest-backups --rep
o1-s3-endpoint=s3.amazonaws.com --repo1-s3-key=<redacted> --repo1-s3-key-secret=<redacted> --repo1-s3-region= --rep
o1-s3-token=<redacted> --repo1-type=s3 --stanza=db1
2022-11-15 22:07:41.504 P00 INFO: stanza-create for stanza 'db1' on repo1
2022-11-15 22:07:43.192 P00 INFO: stanza-create command end: completed successfully (3008ms)
postgres@repository:~$
```

Figure 7.9: pgBackRest stanza creation

Considering we created the stanza for both database servers, we can now get the first backups. By default, **pgBackRest** will get a **full** backup if there is no previous backup set in the stanza, but if it finds an existing **full** backup, then it will take an **incremental** backup unless we use the **--type** flag and specify another backup type (full, differential).

Figures 7.10 the current scenario's backup operations from the first database server:

```
postgres@repository:~$ pgbackrest --stanza=db1 --log-level-console=info backup
2022-11-15 22:55:30.815 P00 INFO: backup command begin 2.41: --exec-id=10577-9694cdb2 --log-level-console=info
--pg1-host=db1 --pg1-host-user=postgres --pg1-path=/var/lib/postgresql/13/main --repo1-path=/pgbackrest --repo1-r
etention-full=2 --repo1-s3-bucket=pgbackrest-backups --repo1-s3-endpoint=s3.amazonaws.com --repo1-s3
-key=<redacted> --repo1-s3-key-secret=<redacted> --repo1-s3-region= --repo1-s3-token=<redacted> --repo1-
type=s3 --stanza=db1 --start-fast
WARN: no prior backup exists, incr backup has been changed to full
2022-11-15 22:55:32.313 P00 INFO: execute non-exclusive pg_start_backup(): backup begins after the requested im
mediate checkpoint completes
2022-11-15 22:55:32.839 P00 INFO: backup start archive = 000000010000000000000001A, lsn = 0/1A000028
2022-11-15 22:55:32.839 P00 INFO: check archive for prior segment 0000000100000000000000019
2022-11-15 22:58:17.586 P00 INFO: execute non-exclusive pg_stop_backup() and wait for all WAL segments to archi
ve
2022-11-15 22:58:17.793 P00 INFO: backup stop archive = 000000010000000000000001A, lsn = 0/1A000138
2022-11-15 22:58:18.282 P00 INFO: check archive for segment(s) 000000010000000000000001A:000000010000000000000001
A
2022-11-15 22:58:19.693 P00 INFO: new backup label = 20221115-225454F
2022-11-15 22:58:21.811 P00 INFO: full backup size = 479.5MB, file total = 1233
2022-11-15 22:58:21.811 P00 INFO: backup command end: completed successfully (170997ms)
2022-11-15 22:58:21.811 P00 INFO: expire command begin 2.41: --exec-id=10577-9694cdb2 --log-level-console=info
--repo1-path=/pgbackrest --repo1-retention-full=2 --repo1-s3-bucket=pgbackrest-backups --repo1-s3-endpoint=s3.
amazonaws.com --repo1-s3-key=<redacted> --repo1-s3-key-secret=<redacted> --repo1-s3-region= --
repo1-s3-token=<redacted> --repo1-type=s3 --stanza=db1
2022-11-15 22:58:22.335 P00 INFO: expire command end: completed successfully (524ms)
postgres@repository:~$
```

Figure 7.10: pgBackRest backup for database server db1

Figure 7.11 show the current scenario's backup operations from the second database server:

```
postgres@repository:~$ pgbackrest --stanza=db2 --log-level-console=info backup
2022-11-15 22:59:19.516 P00 INFO: backup command begin 2.41: --exec-id=11028-bdc7d9ab --log-level-console=info
--pg1-host=db2 --pg1-host-user=postgres --pg1-path=/var/lib/postgresql/14/main --repo1-path=/pgbackrest --repo1-r
etention-full=2 --repo1-s3-bucket=pgbackrest-backups --repo1-s3-endpoint=s3.amazonaws.com --repo1-s3
-key=<redacted> --repo1-s3-key-secret=<redacted> --repo1-s3-region= --repo1-s3-token=<redacted> --repo1-
type=s3 --stanza=db2 --start-fast
WARN: no prior backup exists, incr backup has been changed to full
2022-11-15 22:59:21.570 P00 INFO: execute non-exclusive pg_start_backup(): backup begins after the requested im
mediate checkpoint completes
2022-11-15 22:59:22.094 P00 INFO: backup start archive = 000000010000000000000001A, lsn = 0/1A000028
2022-11-15 22:59:22.094 P00 INFO: check archive for prior segment 0000000100000000000000019
2022-11-15 23:02:11.363 P00 INFO: execute non-exclusive pg_stop_backup() and wait for all WAL segments to archi
ve
2022-11-15 23:02:11.570 P00 INFO: backup stop archive = 000000010000000000000001A, lsn = 0/1A000138
2022-11-15 23:02:12.058 P00 INFO: check archive for segment(s) 000000010000000000000001A:000000010000000000000001
A
2022-11-15 23:02:13.583 P00 INFO: new backup label = 20221115-225932F
2022-11-15 23:02:15.794 P00 INFO: full backup size = 482MB, file total = 1253
2022-11-15 23:02:15.795 P00 INFO: backup command end: completed successfully (176280ms)
2022-11-15 23:02:15.795 P00 INFO: expire command begin 2.41: --exec-id=11028-bdc7d9ab --log-level-console=info
--repo1-path=/pgbackrest --repo1-retention-full=2 --repo1-s3-bucket=pgbackrest-backups --repo1-s3-endpoint=s3.
amazonaws.com --repo1-s3-key=<redacted> --repo1-s3-key-secret=<redacted> --repo1-s3-region= --
repo1-s3-token=<redacted> --repo1-type=s3 --stanza=db2
2022-11-15 23:02:16.358 P00 INFO: expire command end: completed successfully (563ms)
postgres@repository:~$
```

Figure 7.11: pgBackRest backup for database server db2.

With **pgBackRest**, you can also get quick information about the existing backups in the repository by consulting the catalog per *stanza*. Figure 7.12 demonstrates how the catalog looks for the *db1* stanza after the **full** backup:

```

postgres@repository:~$ pgbackrest --stanza=db1 --log-level-console=info info
stanza: db1
  status: ok
  cipher: none

  db (current)
    wal archive min/max (13): 000000010000000000000018/00000001000000000000001A

    full backup: 20221115-225454F
      timestamp start/stop: 2022-11-15 22:54:54 / 2022-11-15 22:57:39
      wal start/stop: 00000001000000000000001A / 00000001000000000000001A
      database size: 479.5MB, database backup size: 479.5MB
      repo1: backup set size: 27.8MB, backup size: 27.8MB
postgres@repository:~$ █

```

Figure 7.12: pgBackRest backup catalog.

Now, let us think that for any reason, we lost the *db1* server, there was an issue with the storage, the data center was lost, or somebody mistakenly deleted all the contents from the **data_directory** (ouch). In this situation, we can recover our system by restoring one of our backups. Figures 7.13, 7.14, and 7.15 illustrates the deletion of the database data, the restore operation, and the again working system.

```

postgres@db1:~$ du -sch /var/lib/postgresql/13/main
513M   /var/lib/postgresql/13/main
513M   total
postgres@db1:~$ psql -c '\l+ pgbench'
                                List of databases
  Name | Owner | Encoding | Collate | Ctype | Access privileges | Size | Tablespace | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
pgbench | pgbench | UTF8 | C.UTF-8 | C.UTF-8 | | 456 MB | pg_default |
(1 row)

postgres@db1:~$ rm -rf /var/lib/postgresql/13/main/*
postgres@db1:~$ du -sch /var/lib/postgresql/13/main
4.0K   /var/lib/postgresql/13/main
4.0K   total
postgres@db1:~$ psql -c '\l+ pgbench'
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL: could not open file "global/pg_filenode.map": No such file or directory
postgres@db1:~$ █

```

Figure 7.13: Deletion of the database data in the *data_directory*.

We can recover the **full** backup easily using the pgBackRest configuration we already have. Figure 7.14 shows the process:

```

postgres@db1:~$ pgbackrest --log-level-console=info --stanza=db1 restore
2022-11-15 23:39:32.034 P00 INFO: restore command begin 2.41: --exec-id=15929-4f9f0233 --log-level-console=info --log-level-file=detail --pg1-path=/var/lib/postgresql/13/main --repo1-host=repository --repo1-host-user=postgres --stanza=db1
2022-11-15 23:39:33.658 P00 INFO: repo1: restore backup set 20221115-225454F, recovery will start at 2022-11-15 22:54:54
2022-11-15 23:42:33.552 P00 INFO: write updated /var/lib/postgresql/13/main/postgresql.auto.conf
2022-11-15 23:42:33.577 P00 INFO: restore global/pg_control (performed last to ensure aborted restores cannot be started)
2022-11-15 23:42:33.578 P00 INFO: restore size = 479.5MB, file total = 1233
2022-11-15 23:42:33.578 P00 INFO: restore command end: completed successfully (181545ms)
postgres@db1:~$ █

```

Figure 7.14: pgBackRest restore of *db1*.

After recovering the **full** backup, we can verify the size of the **data_directory** and connect using **psql** to check the size of the databases. *Figure 7.15* shows the successful result:

```
postgres@db1:~$ date ; du -sch /var/lib/postgresql/13/main
Tue Nov 15 23:43:57 UTC 2022
512M    /var/lib/postgresql/13/main
512M    total
postgres@db1:~$
postgres@db1:~$ psql -c '\l+ pgbench'
```

Name	Owner	Encoding	Collate	Ctype	Access privileges	Size	Tablespace	Description
pgbench	pgbench	UTF8	C.UTF-8	C.UTF-8		456 MB	pg_default	

```
(1 row)
postgres@db1:~$
```

Figure 7.15: PostgreSQL restored from the pgBackRest backup.

Remember, we can perform such full restorations by taking the existing backups and applying the archived WAL files until the latest. But also possible to restore to a specific point in time if we specify the **--type=time** flag and specify a target timestamp.

These and many other options are supported with **pgBackRest**. Worth to take some time and reviewing the official documentation at <https://pgbackrest.org/>.

Barman

Barman is another open-source tool that has been around for a while and has gotten the attention of many projects and solutions. The same as pgBackRest, Barman, can be used as a central repository to back up multiple database servers. It creates and maintains a backup catalog to get a quick insight into the existing backups.

Barman can produce **full** backups, and when configured as **rsync** mode, it can get **incremental** backups, run with parallelism, and ensure data deduplication and network compression to speed up the transmission when taking the backup remotely.

Barman does not support cloud repositories as storage, it can just use local filesystems. However, for continuous archiving, it supports the standard method (copy or file transmission) or WAL streaming, just as a standby replica. Also, you can use custom *hook scripts* to execute customized routines before or after each backup or WAL archive.

Figure 7.16 illustrates a Barman solution to back up two different PostgreSQL servers.

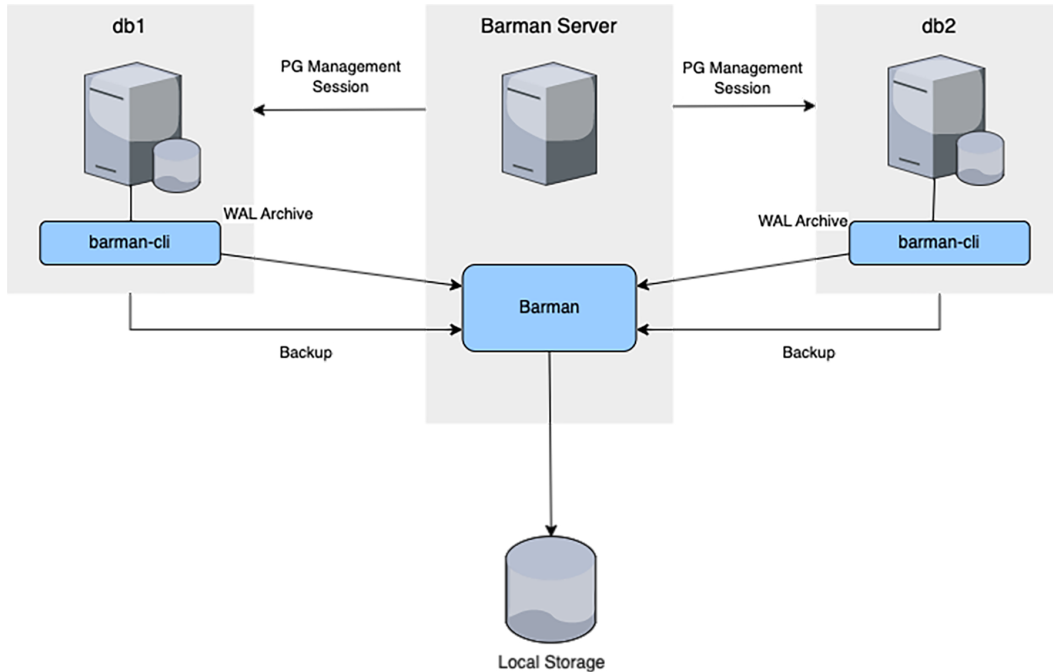


Figure 7.16: Barman for multiple database servers backup.

Using Barman, you need to consider installing the **barman-cli** in the database nodes; this provides an interface for the **archive_command**, so PostgreSQL can push the WAL files for archiving. All the other backup execution happens in the backup server. Also, consider adding the proper configuration to the PostgreSQL servers so the Barman server can connect to **postgres** itself and the Operating System.

The following is an example of the required configuration.

Barman server:

```
barman@repository:~$ cat /etc/barman.conf

[barman]

barman_user = barman

configuration_files_directory = /etc/barman.d

barman_home = /var/lib/barman

log_file = /var/log/barman/barman.log

log_level = INFO
```

```
compression = gzip
```

```
barman@repository:~$ cat /etc/barman.d/ssh-db[1,2].conf
```

```
[ssh-db1]
```

```
description = "PostgreSQL Database db1 (via SSH)"
```

```
ssh_command = ssh postgres@db1
```

```
conninfo = host=db1 user=barman dbname=postgres
```

```
backup_method = rsync
```

```
reuse_backup = link
```

```
backup_options = concurrent_backup
```

```
archiver = on
```

```
-----
```

```
[ssh-db2]
```

```
description = "PostgreSQL Database db2 (via SSH)"
```

```
ssh_command = ssh postgres@db2
```

```
conninfo = host=db2 user=barman dbname=postgres
```

```
backup_method = rsync
```

```
reuse_backup = link
```

```
backup_options = concurrent_backup
```

```
archiver = on
```

db1 and db2 servers:

```
postgres@db1:~$ createuser -s -P barman
```

To get this configuration working you need to allow passwordless SSH from the Barman server to the database servers, from the **barman** user (it runs the barman executable) to the **postgres** user (it runs the postgres service).

```
barman@repository:~$ ssh postgres@db1 'hostname'
```

```
db1
```

```
barman@repository:~$ ssh postgres@db2 'hostname'
db2
```

With all the previous configuration in place, you can get verify all is correctly set and then get the first backups. *Figure 7.17* shows how to verify the configuration from the Barman server for a given database node.

```
barman@repository:~$ barman check ssh-db1
Server ssh-db1:
  PostgreSQL: OK
  superuser or standard user with backup privileges: OK
  wal_level: OK
  directories: OK
  retention policy settings: OK
  backup maximum age: OK (no last_backup_maximum_age provided)
  backup minimum size: OK (0 B)
  wal maximum age: OK (no last_wal_maximum_age provided)
  wal size: OK (0 B)
  compression settings: OK
  failed backups: OK (there are 0 failed backups)
  minimum redundancy requirements: OK (have 0 backups, expected at least 0)
  ssh: OK (PostgreSQL server)
  systemid coherence: OK (no system Id stored on disk)
  archive_mode: OK
  archive_command: OK
  continuous archiving: OK
  archiver errors: OK
barman@repository:~$
```

Figure 7.17: Barman configuration check.

If all the previous checks are OK, you can take a backup. See *Figure 7.18*, which illustrates a backup operation using Barman:

```
barman@repository:~$ barman backup ssh-db1
Starting backup using rsync-concurrent method for server ssh-db1 in /var/lib/barman/ssh-db1/base/20221116T040407
Backup start at LSN: 0/2C000028 (00000003000000000000000000002C, 00000028)
This is the first backup for server ssh-db1
WAL segments preceding the current backup have been found:
00000003000000000000000000001B from server ssh-db1 has been removed
00000003000000000000000000001C from server ssh-db1 has been removed
00000003000000000000000000001D from server ssh-db1 has been removed
00000003000000000000000000001E from server ssh-db1 has been removed
00000003000000000000000000001F from server ssh-db1 has been removed
000000030000000000000000000020 from server ssh-db1 has been removed
000000030000000000000000000021 from server ssh-db1 has been removed
000000030000000000000000000022 from server ssh-db1 has been removed
000000030000000000000000000023 from server ssh-db1 has been removed
000000030000000000000000000024 from server ssh-db1 has been removed
000000030000000000000000000025 from server ssh-db1 has been removed
000000030000000000000000000026 from server ssh-db1 has been removed
000000030000000000000000000027 from server ssh-db1 has been removed
000000030000000000000000000028 from server ssh-db1 has been removed
000000030000000000000000000029 from server ssh-db1 has been removed
00000003000000000000000000002A from server ssh-db1 has been removed
Starting backup copy via rsync/SSH for 20221116T040407
Copy done (time: 18 seconds)
This is the first backup for server ssh-db1
Asking PostgreSQL server to finalize the backup.
Backup size: 484.2 MiB. Actual size on disk: 484.2 MiB (-0.00% deduplication ratio).
Backup end at LSN: 0/2C000138 (00000003000000000000000000002C, 00000138)
Backup completed (start time: 2022-11-16 04:04:07.591253, elapsed time: 24 seconds)
Processing xlog segments from file archival for ssh-db1
000000030000000000000000000028
00000003000000000000000000002C
00000003000000000000000000002C.00000028.backup
barman@repository:~$
```

Figure 7.18: Barman backup operation

As we stated before, Barman includes a backup catalog so that you can get a quick view of the existing backups for a given database server and the status of the server. *Figure 7.19* shows how this catalog views look.

```
barman@repository:~$ barman list-backups ssh-db1
ssh-db1 20221116T040407 - Wed Nov 16 04:02:36 2022 - Size: 484.2 MiB - WAL Size: 16.0 KiB
barman@repository:~$
barman@repository:~$ barman status ssh-db1
Server ssh-db1:
  Description: PostgreSQL Database db1 (via SSH)
  Active: True
  Disabled: False
  PostgreSQL version: 13.9
  Cluster state: in production
  pgspresso extension: Not available
  Current data size: 484.6 MiB
  PostgreSQL Data directory: /var/lib/postgresql/13/main
  Current WAL segment: 000000030000000000000002E
  PostgreSQL 'archive_command' setting: barman-wal-archive repository ssh-db1 %p
  Last archived WAL: 000000030000000000000002D, at Wed Nov 16 04:02:40 2022
  Failures of WAL archiver: 0
  Server WAL archiving rate: 4.75/hour
  Passive node: False
  Retention policies: not enforced
  No. of available backups: 1
  First available backup: 20221116T040407
  Last available backup: 20221116T040407
  Minimum redundancy requirements: satisfied (1/0)
barman@repository:~$
```

Figure 7.19: Barman backup list and server status.

Obviously, with Barman, we can also recover our databases from a backup. Considering we lost the *db1* server, and we were able to rebuild the machine with the same storage paths but empty. We can restore our data and get postgres up and running, as *Figure 7.20* shows:

```
barman@repository:~$ barman recover --remote-ssh-command "ssh postgres@db1" ssh-db1 20221116T040407 /var/lib/postgresql/13/main
Starting remote restore for server ssh-db1 using backup 20221116T040407
Destination directory: /var/lib/postgresql/13/main
Remote command: ssh postgres@db1
Copying the base backup.
Copying required WAL segments.
Generating archive status files
Identify dangerous settings in destination directory.

IMPORTANT
These settings have been modified to prevent data losses

postgresql.auto.conf line 6: archive_command = false

Recovery completed (start time: 2022-11-17 04:37:46.317480+00:00, elapsed time: 24 seconds)
Your PostgreSQL server has been successfully prepared for recovery!
barman@repository:~$
```

Figure 7.20: Barman recovers from the backup server to the remote db1 server.

Consider that the operation shown in *Figure 7.20* required the PostgreSQL service on *db1* was stopped. Also, you can see the **IMPORTANT** message, the **archive_command** was set to false, causing any new archive from this just restored server to fail.

This is expected because this restore might happen against a different database server, such as a spare server. In such a case, the database might completely diverge from the original *db1*. We need to configure a new database server in the Barman configuration to continue backing up the “new” *db1*.

```
barman@repository:~$ cat /etc/barman.d/ssh-db1-new.conf

[ssh-db1-new]

description = "PostgreSQL Database db1-new (via SSH)"

ssh_command = ssh postgres@db1

conninfo = host=db1 user=barman dbname=postgres

backup_method = rsync

reuse_backup = link

backup_options = concurrent_backup

archiver = on
```

Barman has been around in the PostgreSQL since a while, so it has been proved and used in a large number of projects. You can find some other features and details we did not cover in this chapter by looking at its documentation at <https://pgbarman.org/documentation/>.

pg_probackup

We might tell **pg_probackup** is the “youngest” tool from this section, just as the previous ones, it is an open-source project. And, like other open-source projects, it brings some extra functions to extend the flexibility for different types of PostgreSQL implementations.

pg_probackup offers interesting options like incremental restore, skipping those data pages already present in the **data_directory** that have not changed from the ones in the backup, automatic and on-demand data consistency checks, backup merge, and deduplication.

It also supports other features similar to the previous tools, like parallelism, remote operations, backup from a standby, backup catalog, and more.

Figure 7.21 shows what a setup of **pg_probackup** backing up two different PostgreSQL servers looks like:

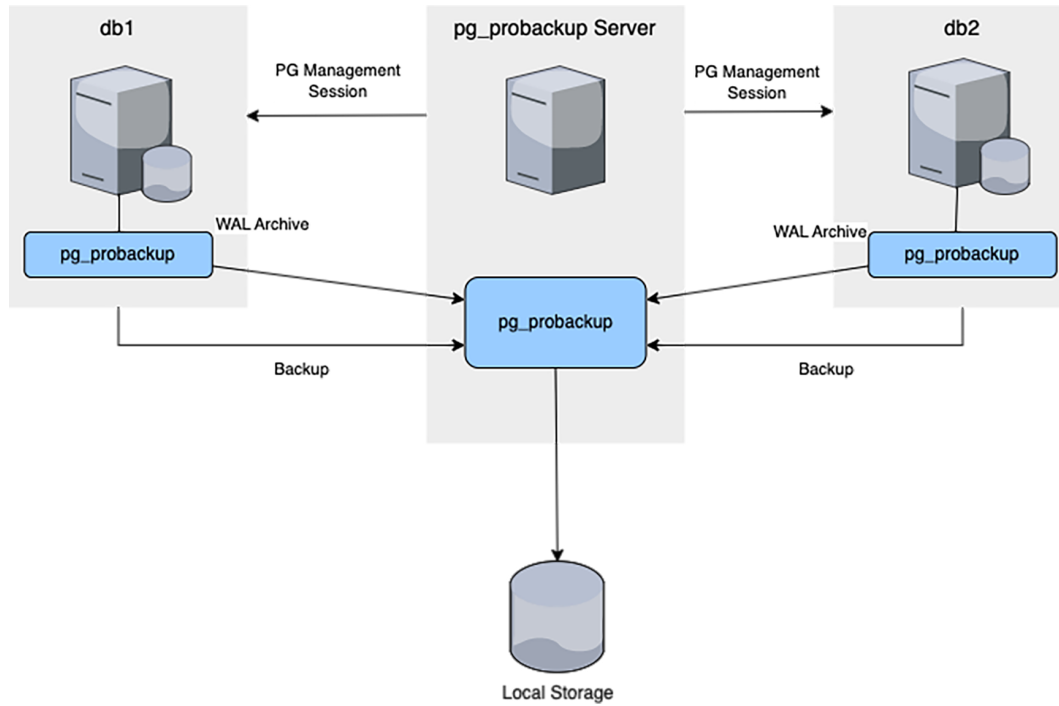


Figure 7.21: *pg_probackup* for multiple database servers backup.

Different from the previous tools, **pg_probackup** does not use configuration files, or well, it does but accordingly to the documentation, these should not be edited manually. Rather, you need to configure your setup using the different command options included with the tool.

As illustrated in Figure 7.21, you need to install the tool in the backup server and the database servers, and you have to install the corresponding package for the PostgreSQL version. Since we are using the same *db1* and *db2* servers from the previous examples, one is version 13, and the other is version 14, you need to install the packages accordingly, two in the backup server and the corresponding single one in the database servers.

Once installed, the next step is to create the backup repository in the local system of the backup server and initialize the catalog on it. Figure 7.22 shows this action:

```

postgres@repository:~$ pg_probackup-13 init -B /pg_probackup/db1
INFO: Backup catalog '/pg_probackup/db1' successfully initied
postgres@repository:~$
postgres@repository:~$ pg_probackup-14 init -B /pg_probackup/db2
INFO: Backup catalog '/pg_probackup/db2' successfully initied

```

Figure 7.22: pg_probackup catalog init.

The next step is to add the instances to be backed up, in this case, two remote instances. See Figure 7.23, which illustrates this point:

```

postgres@repository:~$ pg_probackup-13 add-instance -B /pg_probackup/db1 -D /var/lib/postgresql/13/main --instanc
e db1 --remote-host=db1 --remote-user=postgres --remote-path=/usr/bin
INFO: Instance 'db1' successfully initied
postgres@repository:~$
postgres@repository:~$ pg_probackup-14 add-instance -B /pg_probackup/db2 -D /var/lib/postgresql/14/main --instanc
e db2 --remote-host=db2 --remote-user=postgres --remote-path=/usr/bin
INFO: Instance 'db2' successfully initied
postgres@repository:~$

```

Figure 7.23: pg_probackup adding remote instances.

Just as with the other two tools, to perform the remote backup operations, we need to configure the password-less SSH between the backup server and the database servers. Also, since the continuous archive is required, we must set the **archive_command** properly. These details are deeply covered in the online documentation of the tool.

Now, considering we have set the previous details, we can get the backup from the remote instances. Figure 7.24 shows the backup operation from the **db2** instance:

```

postgres@repository:~$ pg_probackup-14 backup -B /pg_probackup/db2 --instance db2 -b FULL
INFO: Backup start, pg_probackup version: 2.5.8, instance: db2, backup ID: RLIT51, backup mode: FULL, wal mode: A
RCHIVE, remote: true, compress-algorithm: none, compress-level: 1
WARNING: This PostgreSQL instance was initialized without data block checksums. pg_probackup have no way to detec
t data block corruption without them. Reinitialize PGDATA with option '--data-checksums'.
INFO: Database backup start
INFO: wait for pg_start_backup()
INFO: Wait for WAL segment /pg_probackup/db2/wal/db2/0000000100000000000000027 to be archived
INFO: PGDATA size: 486MB
INFO: Current Start LSN: 0/27000028, TLI: 1
INFO: Start transferring data files
INFO: Data files are transferred, time elapsed: 16s
INFO: wait for pg_stop_backup()
INFO: pg_stop_backup() successfully executed
INFO: stop_lsn: 0/28001E48
INFO: Wait for LSN 0/28001E48 in archived WAL segment /pg_probackup/db2/wal/db2/0000000100000000000000028
INFO: Getting the Recovery Time from WAL
INFO: Syncing backup files to disk
INFO: Backup files are synced, time elapsed: 0
INFO: Validating backup RLIT51
INFO: Backup RLIT51 data files are valid
INFO: Backup RLIT51 resident size: 486MB
INFO: Backup RLIT51 completed
postgres@repository:~$

```

Figure 7.24: pg_probackup backup from remote PostgreSQL.

With **pg_probackup**, we can check the backup catalog and get information about the existing backup sets. *Figure 7.25* demonstrates this operation:

```
postgres@repository:~$ pg_probackup-14 show -B /pg_probackup/db2
```

BACKUP INSTANCE 'db2'

Instance	Version	ID	Recovery Time	Mode	WAL Mode	TLI	Time	Data	WAL	Zratio	Start LSN	S
top LSN	Status											
db2	14	RLIT51	2022-11-18 01:52:15+00	FULL	ARCHIVE	1/0	22s	486MB	16MB	1.00	0/27000028	0
/28001E48	OK											

```
postgres@repository:~$
```

Figure 7.25: pg_probackup backup catalog

Finally, and as expected, we can also recover from a backup in case of a catastrophe, or if we want to create a new server with a copy of the data. See *Figure 7.26*, which illustrates a restore for the **db2** server:

```
postgres@repository:~$ pg_probackup-14 restore -B /pg_probackup/db2 --instance db2
```

INFO: Validating backup RLIT51
 INFO: Backup RLIT51 data files are valid
 INFO: Backup RLIT51 WAL segments are valid
 INFO: Backup RLIT51 is valid.
 INFO: Restoring the database from backup at 2022-11-18 01:51:01+00
 INFO: Start restoring backup files. PGDATA size: 486MB
 INFO: Backup files are restored. Transferred bytes: 486MB, time elapsed: 18s
 INFO: Restore incremental ratio (less is better): 100% (486MB/486MB)
 INFO: Syncing restored files to disk
 INFO: Restored backup files are synced, time elapsed: 3s
 INFO: Restore of backup RLIT51 completed.
 postgres@repository:~\$

Figure 7.26: pg_probackup restore from a backup.

There is no doubt that **pg_probackup** is another great tool we can consider when designing our backup strategies. It has some other features we did not cover in this chapter, take a look at the GitHub to know more on the same.

Conclusion

As we saw, PostgreSQL provides all the required features to protect our databases against losses. We can back up our databases in two ways, logically and physically. There are some tools and techniques already present in the community packages of postgres, but also other open-source projects bring extra features.

In the next chapter, we will learn about PostgreSQL replication.

Bibliography

- pg_basebackup: <https://www.postgresql.org/docs/15/app-pgbasebackup.html>
- Point In Time Recovery: <https://www.postgresql.org/docs/15/continuous-archiving.html>

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>

